

Module 11 — Stretch-Thu: Distributed Observability Router

Honors Track. This is an Honors Track stretch assignment — not required for program completion, but required for Honors distinction. Stretch is for learners who have completed all core assignments, are On Track or Advanced, and are attending consistently.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: Module 11 Applied Lab and Integration Task. This stretch decomposes the Module 10 backend into a multi-service topology and reuses the M11 instrumentation discipline across all of them. No challenge tier prerequisite. **Pair collaboration is permitted** (split the `router` service from one backend across two learners); encouraged, not required.

What success looks like: By the end of this stretch, you can decompose a monolithic backend into three coordinated FastAPI services, compose them with Docker Compose on a shared network, expose per-service `/metrics`, author a routing classifier that achieves at least 0.80 accuracy on a held-out fixture, propagate a per-request correlation id across service boundaries, consolidate per-service metrics and per-decision logs into a multi-service evaluation report, and articulate the monolith-vs-microservice trade-off for this stack.

Objective

Decompose the Module 10 backend into three coordinated services — a `ner-kg` service (handling `/extract` and `/kg/query`), a `rag` service (handling `/rag/answer`), and a thin `router` service that classifies incoming questions and forwards each to the appropriate backend. Each service exposes its own `/metrics`. The router generates a shared **request-id** per inbound request that flows across service boundaries and appears in every downstream log line. An `analyze_routing.py` script consolidates the per-service metrics and per-decision logs into a multi-service evaluation report.

This stretch is the multi-agent scaffold for the capstone. The router’s classification step is the equivalent of an agent’s “tool selection”; the per-decision logs + cross-service correlation are the observability layer that makes such an orchestrator debuggable in production.

Setup

Use the standalone template repo:

Stretch-Thu template repo: [LevelUp-Applied-AI/ml1-s211](#). Click “Use this template” → “Create a new repository” on the template page, name your copy `ml1-s211`, leave Visibility as Public, and create the repository under your own GitHub account.

If you have not already installed the Module 11 packages, go back to the [Setup: install before the concept session](#) section of the reading. This stretch uses Docker Compose and `httpx` (already covered in the Module 10 / 11 stack).

```
git clone https://github.com/<your-username>/ml1-s211.git
cd ml1-s211
git checkout -b stretch-thu-observability-router
python3.11 -m venv .venv          # use 3.11 - pydantic 2.6 does not build on 3.12
source .venv/bin/activate
pip install -r requirements.txt
```

Learning outcomes

By completing this stretch you will be able to:

- Decompose a monolithic backend into per-domain microservices and articulate the trade-off for *this* stack.
- Compose a three-service Docker Compose topology (`ner-kg` , `rag` , `router`) on a shared network with per-service `/metrics`.
- Generate a per-request correlation id at the router and propagate it through downstream calls so logs across services can be joined.
- Author a routing classifier and quantitatively evaluate its accuracy on a held-out fixture.
- Consolidate per-service metrics + per-decision logs into a single multi-service evaluation report.

Tasks

1. Build the three services

- `services/ner_kg/app.py` — FastAPI service exposing `/extract` , `/kg/query` , and `/metrics` .
- `services/rag/app.py` — FastAPI service exposing `/rag/answer` and `/metrics` .
- `services/router/app.py` — FastAPI service exposing `/route` and `/metrics` . The router classifies the inbound question, generates a request-id, forwards the request to the chosen backend with the request-id in a header, and emits a structured per-decision log line.

2. Compose the three-service stack

`docker-compose-stretch.yml` declares the three services on a shared network. Healthchecks on each. The router depends on both backends being healthy.

3. Author the routing classifier

`services/router/` contains the classifier. The held-out fixture lives at `services/router/fixtures/routing_questions.json` . The classifier must achieve **routing accuracy ≥ 0.80** on the fixture.

4. Cross-service correlation

The router generates a request-id (`uuid4` is fine) and forwards it as the `X-Request-ID` header (same canonical spelling as the Reading, Lab, and Drill — HTTP header names are case-insensitive in transit, but consistency matters for readability and for any string-equal comparison in tests or log filters). The backends echo it on every log line they emit for that request. Verify by `docker compose logs` filtered to one request-id and seeing a continuous trace across services.

5. Multi-service evaluation report

`analyze_routing.py` reads each service’s `/metrics` plus the router’s per-decision logs and writes `routing-analysis-report.md` . The report includes: per-service request counts and error rates, routing accuracy on the fixture, the distribution of routing decisions across backends, and any cross-service correlation findings.

The autograder imports five function names from `analyze_routing.py` — fill in their bodies (the starter ships docstrings + signatures), do not rename them:

- `load_fixture(path) -> list[dict]` — read `routing_questions.json` and return its list of questions.
- `drive_router(router_base, questions) -> list[dict]` — POST each question to `{router_base}/route` and return the per-question routing decisions.
- `routing_accuracy(decisions) -> float` — fraction of decisions whose `target` matches the question’s `expected` .
- `fetch_metrics(base_url) -> str` — GET `{base_url}/metrics` and return the OpenMetrics text body.
- `render_report(decisions, accuracy, service_metrics, report_path) -> None` — write the Markdown report at `report_path` with the four required sections (`## Routing Accuracy` , `## Per-Service Metrics` , `## Routing Pattern` , `## Cross-Service Correlation`).

Each function must have a real body — the autograder rejects unmodified stubs (`raise NotImplementedError` , `pass` , `return None` , or `return <Constant>`).

6. Architecture write-up

`architecture.md` is a one-page write-up of the monolith-vs-microservice trade-off as it applies to *this* stack — what the three-service split bought you, and what it cost.

Running the full pipeline

```
docker compose -f docker-compose-stretch.yml up --build -d
```

The compose binds:

- `ner-kg` → `http://localhost:8101`
- `rag` → `http://localhost:8102`
- `router` → `http://localhost:8100`

Send a request through the router:

```
curl -X POST http://localhost:8100/route \
-H 'content-type: application/json' \
-d '{"question": "Who is the CEO of OpenAI?"}'
```

Run the routing analysis:

```
python analyze_routing.py \
--fixture services/router/fixtures/routing_questions.json \
--router-base http://localhost:8100 \
--ner-kg-base http://localhost:8101 \
--rag-base http://localhost:8102 \
--report-out routing-analysis-report.md
```

Tests

```
pytest tests/ -v
```

The autograder gates:

- `docker-compose-stretch.yml` defines exactly three services with the expected names.
- Each service exposes `/metrics` returning OpenMetrics text format.
- The router classifies the held-out fixture with accuracy ≥ 0.80.
- `analyze_routing.py` runs end-to-end and produces `routing-analysis-report.md` .

Submission

- Commit your changes to `stretch-thu-observability-router` and push.
- Open a PR within your fork against `main` .
- Confirm the autograder workflow succeeds.

Pair-collaboration submission. If you worked as a pair, both learners submit independently — there is **one shared PR**, but each learner pastes the same PR URL into their own TalentLMS slot so each receives a grade. Both learners must list the other in the PR description (name + GitHub username + the part of the work each learner owned). Grading evaluates the PR’s deliverable, not the per-learner contribution: both pair members receive the same score, and the collaborator-attribution paragraph is the audit trail.

Your PR description must include:

- A one-paragraph summary of your service decomposition and routing approach.
- The routing accuracy you achieved on the held-out fixture.
- A short note on cross-service request-id correlation (how you implemented it and how you verified it works).
- If pair-collaborated, the name and GitHub username of your collaborator and how you split the work.
- Paste your PR URL into TalentLMS → Module 11 → Stretch-Thu 11 to submit this assignment. **Pair learners: each paste the same PR URL into your own TalentLMS slot.**

